

Async JavaScript: Promises, async/await

◆ What is Async JavaScript?

JavaScript is single-threaded and uses an event loop to handle asynchronous operations like network requests, timers, and I/O without blocking the main thread.

◆ Promises

A **Promise** represents a value that may be available now, later, or never.

- Pending
- Fulfilled
- Rejected

```
fetch('/api/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

◆ async / await

The `async` keyword declares a function that returns a Promise. The `await` keyword pauses execution until the Promise resolves.

```
async function getData() {
  try {
    const response = await fetch('/api/data');
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error('Error fetching:', error);
  }
}
```

◆ Retry Mechanism

Retrying failed requests is critical in flaky network environments. Here's how to retry a request up to 3 times:

```
async function retryRequest(url, retries = 3) {
  for (let i = 0; i < retries; i++) {
    try {
      const res = await fetch(url);
      if (!res.ok) throw new Error('Request failed');
      return await res.json();
    } catch (err) {
      console.warn(`Retry ${i + 1} failed`);
      if (i === retries - 1) throw err;
    }
  }
}
// Instead of url we can pass a function and utilize it as a utility function
// for all API calls made in the application
```

 **Tip:** Add exponential backoff using `setTimeout` for better results.

◆ Real Use Cases

- Parallel API calls with `Promise.all`
- Abort long requests with `Promise.race`
- Streaming data using async generators

◆ Interview Tips

- Use `try/catch` in `async/await` for error handling
- Avoid sequential `await` where `Promise.all` is better
- Understand how the event loop impacts async behavior

◆ Interview Questions

Basic

- What is asynchronous JavaScript?
- What is a Promise?
- What is the difference between callbacks and Promises?
- How do you handle errors in Promises?
- What is the difference between `then` and `catch`?

Intermediate

- What is `async/await`?
- How does `async/await` simplify Promises?
- What happens if a Promise inside an `async` function fails?
- What is `Promise.all` and `Promise.race`?
- Can we use `await` outside an `async` function?

Advanced

- What are `async` generators and how do they work?
- How to implement retry logic for failed requests?
- How does the event loop handle `async/await`?
- When should we prefer `Promise.all` over sequential `await`?
- How can you abort a fetch call with a timeout using `Promise.race`?